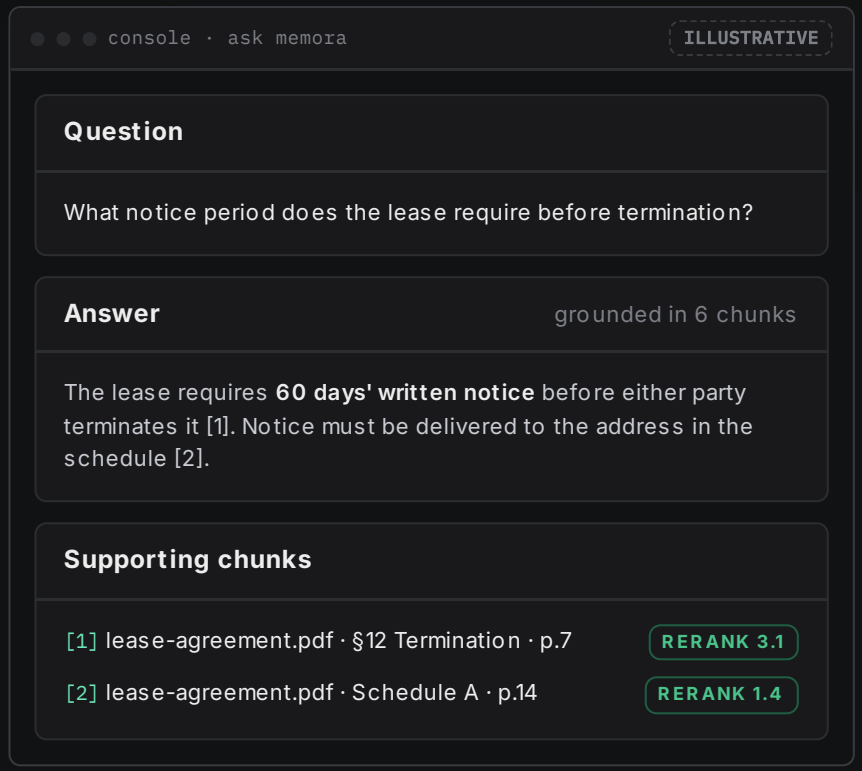
Memora

OCTOBER 2026

PRODUCT HANDBOOK

Durable, retrievable memory for applications that need context.

What Memora is, who it serves, how a document becomes a cited answer, and how its parts fit together.



Product
Memora · memory and RAG service

Evidence
Memora repository, main @ f4c6b8c

Audience
Product, integration and engineering

Product views
Illustrative, drawn from the console's design

HANDBOOK

Contents

Nine sections, from what Memora is to how it is built. Each page covers one idea.

01 **Product overview** 3
- What Memora is 3
- Who it serves and what it solves 4

02 **Core concepts** 5
- The ownership chain 5

03 **Features** 6
- Capability map 6
- Registering sources 7
- Extraction Agent: reading documents 8
- Chunking 9
- Embeddings and strategies 10
- Ask Memora: retrieval and grounded answers 11
- The knowledge graph 12
- Graph RAG and the Arena 13
- Benchmarks, Advisor and Source inspector 14
- Unified Graph RAG 15
- Google Drive connector 16
- The operator console 17
- Evaluation harness 18

04 **User workflows** 19
- Getting started as an integrating application 19

05 **Architecture** 20
- System map 20
- Component responsibilities 21
- Sequence: an application asks a question 22

06 **Data flows & integrations** 23
- How information is stored 23
- Integrations and environments 24
- Integration example: Pingie on Memora 25

07 **Security & permissions** 26
- Trust boundaries 26
- Security controls 27

08 **Limitations & scope** 28
- What is built, partial, or not built 28

09 **Quick reference** 29
- Quick reference 29
- Flashcards 30
- Glossary 31

01 · PRODUCT OVERVIEW

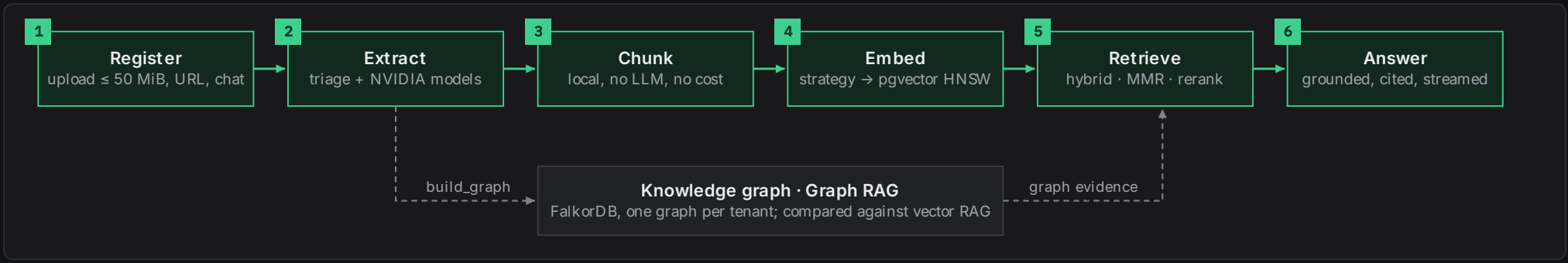
What Memora is

01

Memora is a memory service for other applications. An application registers files, URLs or chat transcripts in a workspace; Memora stores them, reads them with NVIDIA models, splits them into chunks, embeds and indexes them, builds a knowledge graph, and answers questions with cited evidence, scoped so each application sees only its own data.

FIG 01 **From upload to answer**

each stage runs on request or after the one before it



One ownership chain

Tenant → application → subject → source. Every row knows its owners, so nothing is reachable by ID alone.

Reads hard documents

Per-page triage sends scans, tables and charts to layout and vision models; clean text stays local and free.

Cited, honest answers

Hybrid search, reranking and a grounded prompt; the answer says when the knowledge is insufficient.

Graph RAG, measured

A knowledge graph per tenant, and an Arena and benchmarks that compare it with vector RAG on your own questions.

KEY TAKEAWAY

Memora is infrastructure, not an end-user app: client applications such as Pingie call its API, and operators manage it from the console.

01 · PRODUCT OVERVIEW

Who it serves and what it solves

Teams whose applications must remember documents and conversations, without each building its own retrieval stack.

PROBLEM → HOW MEMORA RESPONDS

PROBLEM	RESPONSE
Every application rebuilds parsing, chunking, embeddings and search.	One API covers registration through grounded answers; an application only sends files and questions.
Simple parsers lose tables, scans and charts.	Hard pages are routed to NVIDIA layout and vision models; every run is versioned with its cost.
Customer data must never cross between tenants.	A credential resolves to exactly one application; every service call checks ownership and answers 404 otherwise.
Nobody knows whether Graph RAG is worth its cost.	The Arena and benchmarks score vector, graph and hybrid pipelines on the same questions.

Who uses which part

	USES
Client applications	The REST API with a `memora_…` token
Operators	The console, signed in with Google
Integrators	The public site's API reference

How to read this handbook

Status reflects the repository at f4c6b8c (19 Oct 2026); numbers are `config.py` defaults. The repository has no screenshots, so product views are labelled mocks.

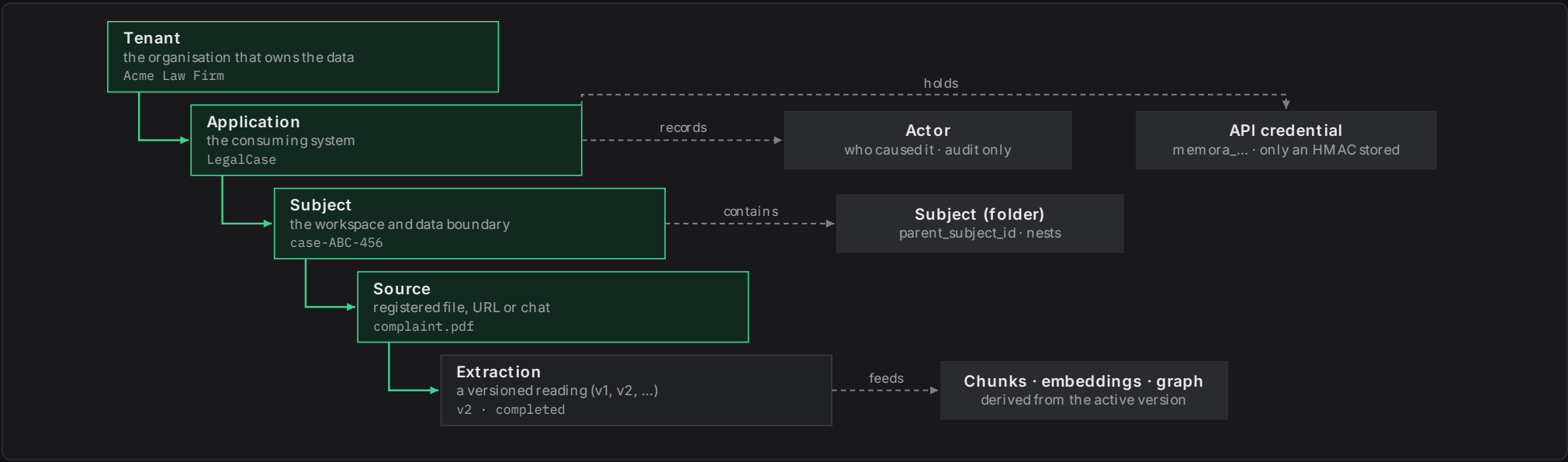
- IMPLEMENTED — Verified in the source code.
- PARTIAL — Works, with narrower scope than described.
- API ONLY — Built; reachable through the API only.
- NOT BUILT — Designed or named; no code yet.

02 · CORE CONCEPTS

02 The ownership chain

Everything in Memora hangs off one chain. Reading it top to bottom is the whole data model.

FIG 02 **The ownership chain** README · db/models



An actor is not a subject

The actor caused a workspace to exist; the subject is the workspace.

Ownership is derived

A source's tenant and application are copied from its subject, never sent by the client.

The chain must agree

A source asked for under the wrong subject is not found, even when you own both.

Not an identity provider

An actor stores only your `external_id`: no email, password or role.

03 · FEATURES

03 Capability map

Everything Memora does, grouped as the console groups it. Status shows what the repository implements.

Ownership & folders
IMPLEMENTED

Tenants, applications, actors, subjects nested as folders, with ownership checked on every call.

Source registration
IMPLEMENTED

Upload up to 50 MiB, or register a URL or chat; move, stream and delete files.

Extraction Agent
IMPLEMENTED

Per-page triage, NVIDIA layout, vision and text models, versioned runs with cost.

Chunking
IMPLEMENTED

Local, structure-aware chunks of 300–500 tokens with section paths; no model cost.

Embeddings
IMPLEMENTED

Immutable strategies over Nemotron embeddings in pgvector, with coverage and retries.

Ask Memora
IMPLEMENTED

Hybrid retrieval, MMR, reranking and a grounded, cited, streamed answer.

Knowledge graph
IMPLEMENTED

Ontology-typed entities and facts in FalkorDB, one graph per tenant, with provenance.

Graph RAG Arena
IMPLEMENTED

Vector, graph and hybrid answers side by side, five graph techniques.

Benchmarks & Advisor
IMPLEMENTED

Score pipelines on question sets; the Advisor weighs accuracy, cost and explainability.

Unified Graph RAG
API ONLY

A rebuilt graph over retrieval chunks, with atomic facts, communities and checked citations.

Google Drive
IMPLEMENTED

Ask Drive PDFs with nothing stored, or import them into the knowledge base.

Evaluation harness
IMPLEMENTED

Generate, review and run question sets; Recall@k, MRR, nDCG, refusals, latency, cost.

03 · FEATURES

Registering sources

A source is one piece of registered knowledge inside a subject. Files are stored; URLs and chats can be recorded as metadata only.

IMPLEMENTED

Upload a file

Any type up to 50 MiB. `extract=true` starts reading straight away; `build_graph=true` adds the graph.

IMPLEMENTED

Register a URL or chat

A row that points at content stored elsewhere; no bytes move.

IMPLEMENTED

Folders

Subjects nest by parent; moves and subtree deletes are checked with recursive queries.

Safe write and delete order

01 **Check scope**
The subject must belong to the caller.

02 **Write bytes**
Original file to local disk or Cloudflare R2.

03 **Write the row**
If it fails, the bytes are removed again.

PURPOSE

Give every file a tracked, owned identity before any processing.

HOW IT BEHAVES

The client never sends tenant or application ids; they are copied from the subject in the path.

WHY IT MATTERS

A caller cannot attach a file to data it does not own.

LIMITS

No file-type allow-list; the size limit applies only when the upload's size is known.

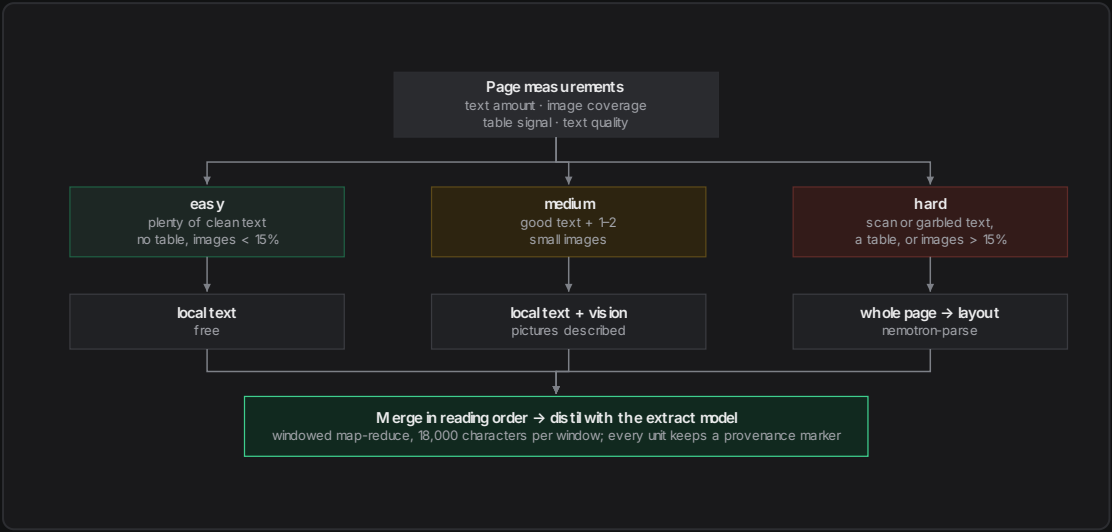
03 · FEATURES

Extraction Agent: reading documents

Local parsers decide which pages are hard; only hard parts go to NVIDIA models. Each run is a new version, with every model call and its cost recorded.

FIG 03 **Per-page triage**

PDF pages; other types route by kind



ROLE	DEFAULT MODEL
layout	`nvidia/nemotron-parse`
vision	`nemotron-3-nano-omni-30b`
extract · answer · graph	`nemotron-3-super-120b`

- Reads PDF, DOCX, PPTX, XLSX, images and plain text.
- **Deep** mode sends every PDF page to the layout model.
- Up to 30 pages per document; 4 calls in parallel by default.
- Fallback: layout → vision → local text; some units failed means `partial`.

KEY TAKEAWAY

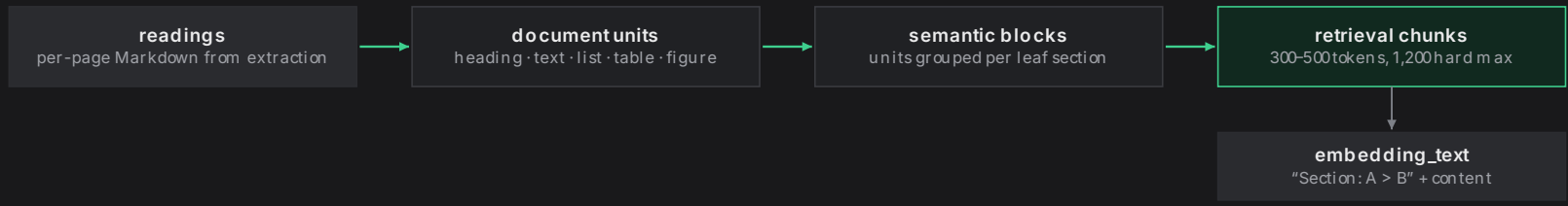
Extraction keeps the per-page readings and the merged text, so chunking and the graph never pay to read the document again.

03 · FEATURES

Chunking

Chunking is local and free. It turns an extraction's readings into retrieval-sized pieces that follow the document's own structure.

FIG 04 **Chunking stages** app/chunking



Target size	Soft max	Hard max	Overlap
300–500 tokens	**600** tokens	**1,200** tokens	**0** by default

PURPOSE

Make pieces that answer one thing each, with their place in the document.

HOW IT BEHAVES

Runs after every successful extraction, or on `POST /sources/{id}/rechunk`. A new version becomes active and older ones switch off in one transaction.

WHY IT MATTERS

Chunks can be rebuilt at will, without model cost, and every answer can name its section.

LIMITS

A coherent section under 300 tokens stays small: chunks are never padded.

03 · FEATURES

Embeddings and strategies

Each chunk is embedded into pgvector under a strategy, indexed with HNSW, and kept in step with the chunks.

	MODEL	STRATEGY
What it is	A physical embedding model	How Memora uses it
Example	`nvidia/nemotron-3-embed-1b`, 2,048 dims	“Memora Dense v1”, seeded by migration
Holds	Dimensions, normalisation	Input field, templates, normalisation, metric
Changes	—	Immutable; a different setup is a new strategy

KEY TAKEAWAY

Because strategies never change, every vector knows exactly what produced it, and several strategies can be compared side by side.

Idempotent

A hash of strategy, model and exact text: unchanged content re-embeds as a no-op and spends nothing.

Tracked and retried

Each vector goes pending → processing → completed or failed, retried up to 3 times with the error kept.

Coverage you can see

The Embeddings page shows total, embedded and processing chunks, coverage, and the strategies.

03 · FEATURES

Ask Memora: retrieval and grounded answers

A question is answered only from chunks the caller owns. Two searches are fused, diversified and reranked; the model then answers from the best six, citing them.

FIG 05 **Query pipeline** default sizes from config.py



console · ask memora ILLUSTRATIVE

Answer grounded in 6 chunks

The lease requires **60 days' written notice** before either party terminates it [1]. Notice must be delivered to the address in the schedule [2].

Supporting chunks

[1] lease-agreement.pdf · §12 Termination · p.7 RERANK 3.1

[2] lease-agreement.pdf · Schedule A · p.14 RERANK 1.4

1. **Scope it**
   Optional subject, source or strategy narrows the search; none means everything you own.
2. **See every stage**
   The response traces how many candidates each stage kept, with similarity and rerank scores.
3. **Stream it**
   `/query/stream` sends the evidence first, then answer text, then token usage.

SCOPE NOTE

The answer is generated by the LLM from the chunks; whether each sentence is supported is not checked on this path. The unified Graph RAG pipeline does check citations.

03 · FEATURES

The knowledge graph

An NVIDIA model reads each extraction for entities and relationships from a fixed ontology. Facts land in one FalkorDB graph per tenant, each pointing back to its source and chunk.

FIG 06 **Knowledge graph build** app/graph



Graph writes happen after the PostgreSQL commit; a startup sweep repairs anything missed.

ONTOLOGY V1 · 10 ENTITY TYPES

PERSON · ORGANIZATION · COMPANY · PRODUCT · LOCATION · TECHNOLOGY · PROJECT · DOCUMENT · CONCEPT · EVENT

20 RELATION TYPES, E.G.

WORKS_FOR · MANAGES · OWNS · USES · SUPPLIES · PARTY_TO · LOCATED_IN · PART_OF · DEPENDS_ON · RELATED_TO

Unknown types fall back to `CONCEPT` and `RELATED_TO`, keeping the original label.

PURPOSE	HOW IT BEHAVES
Capture who and what documents mention, and how they relate.	Built on request per source. A subject's view is filtered to facts from its own sources and subfolders. Retrieval follows 1–3 deterministic hops.
WHY IT MATTERS	**LIMITS**
Relationship questions can use connected facts, not just similar text.	Optional: without a FalkorDB URL, graph endpoints answer 503 and everything else works.

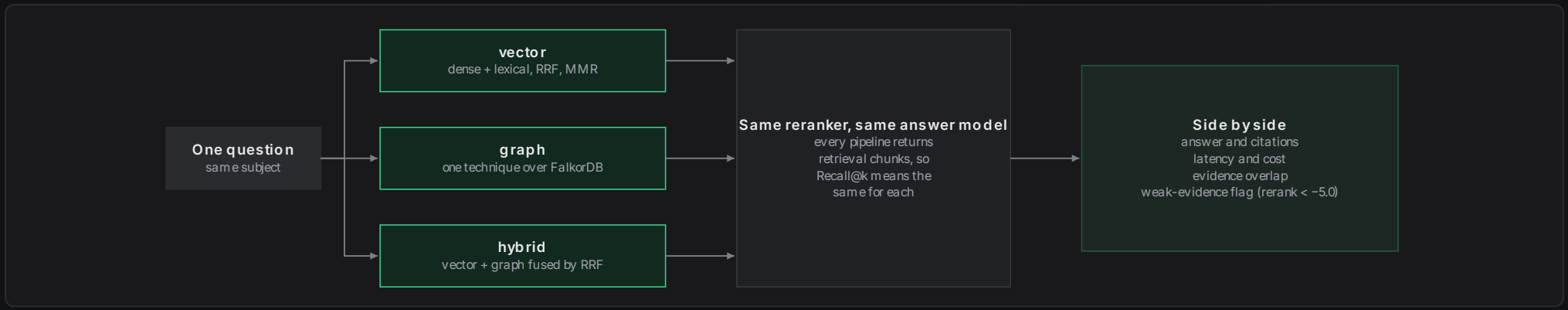
03 · FEATURES

Graph RAG and the Arena

Graph RAG runs as its own service beside vector RAG. The console's Arena asks one question of each pipeline and shows the answers, evidence, time and cost side by side.

FIG 07 **Arena pipelines**

docs/graph-rag-compare.md



FIVE GRAPH TECHNIQUES

spreading
Multi-seed spreading activation with hop decay.

ppr
Personalised PageRank around the question.

dual_level
LightRAG-style entity and theme matching.

paths
Shortest paths between seed entities, as chains.

community
Question against community summaries.

KEY TAKEAWAY

Embedding similarity can't tell a match from a near-miss here, so “nothing relevant” is decided by the reranker: a best score under −5.0 is flagged as weak evidence.

03 · FEATURES

Benchmarks, Advisor and Source inspector

Three tools turn the comparison into evidence: score pipelines on many questions, weigh the trade-offs, and check how one file's graph was built.

Benchmarks — /compare/benchmarks

- Runs pipelines over a question set, loaded from a file.
- Scored against expected answers and supporting chunks.
- Scorecard, per-question wins and losses, results by question type.
- Shows set-up cost: Graph RAG spends effort before any question.

Advisor — reads a benchmark

- Weights accuracy, cost per question and explainability.
- Compares vector RAG with the best graph pipeline per question type.
- Says whether a lead is bigger than the noise for that many questions.
- Explainability is a property of the method, not a measurement.

Source inspector — /compare/inspect

- One file's graph beside the passages it came from.
- Each passage shows the entities and facts it produced.
- Flags weak extraction: empty passages, isolated entities, low-confidence facts.
- Shows only what that file contributed.

Graph explorer

Browse a subject's graph; double-click an entity to pull in its neighbours. Larger circles are mentioned in more passages.

Unified pipeline in benchmarks

The API adds it with `include_unified`, with citation and unsupported-number rates.

API ONLY

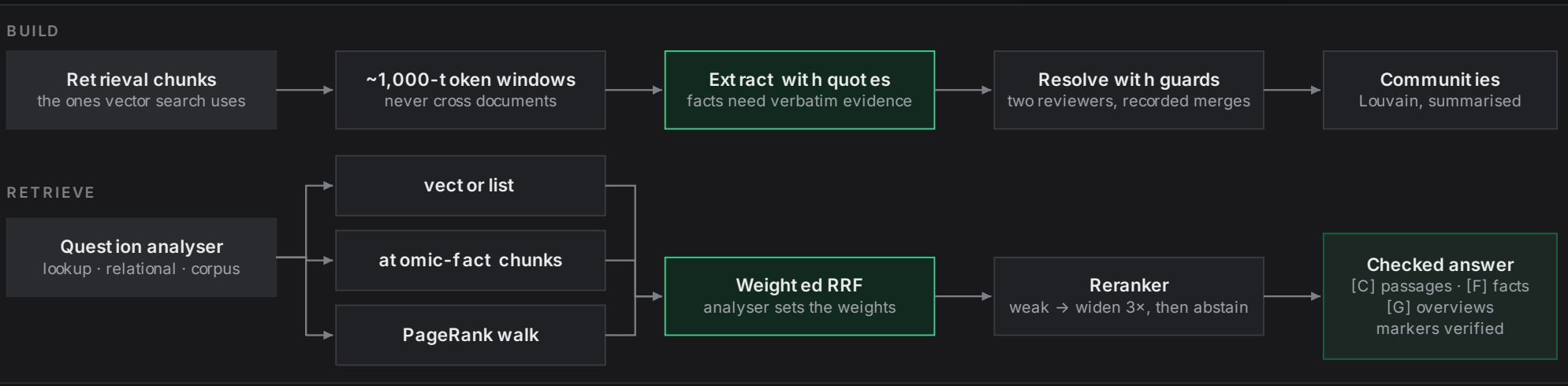
03 · FEATURES

Unified Graph RAG

A second graph build reads the same chunks vector search uses, so every fact cites a real chunk. Its answers carry markers that are checked, never rewritten.

FIG 08 **Unified Graph RAG**

app/graphrag · build and retrieve



PURPOSE	HOW IT BEHAVES	WHY IT MATTERS	LIMITS
Make graph evidence exactly citable and safer to answer from.	Facts need a verbatim quote; merges need a reason and are recorded. If evidence stays weak after widening, it abstains and lists the closest passages.	Answers point to specific chunks, facts and overviews; uncited sentences and unsupported numbers are flagged.	Reached only through the API (`pipeline: "unified"`, `/graph-rag/build`). Measured on one small corpus; community summaries and the entity judge are not cost-metered.

KEY TAKEAWAY

Builds are planned per source (new, changed, outdated, orphaned) and one run per subject holds a lease in PostgreSQL, so it works across workers.

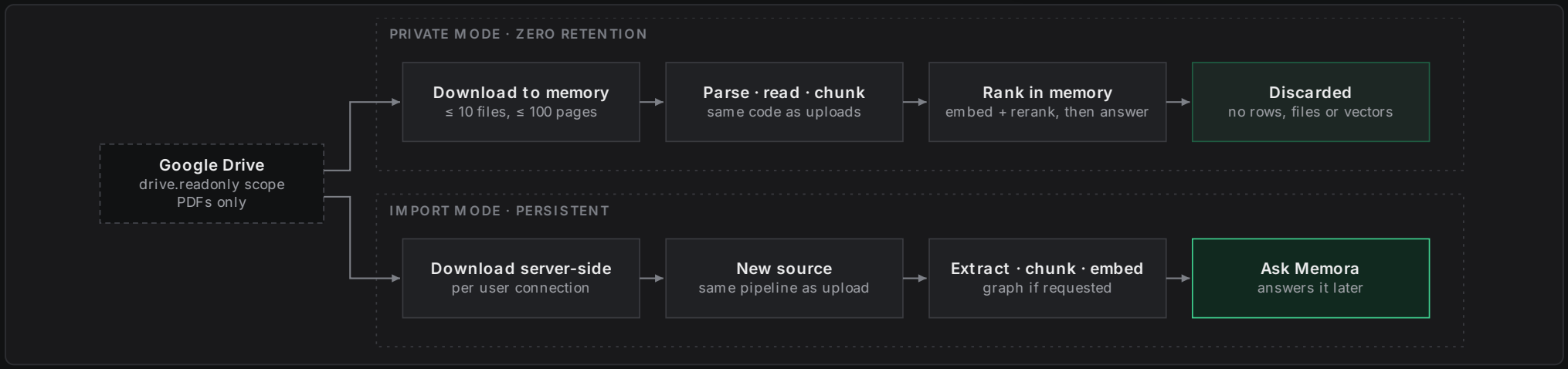
03 · FEATURES

Google Drive connector

A console user connects Drive with read-only access. PDFs can then be asked about without storing anything, or imported into the knowledge base.

FIG 09 **Two modes**

app/connectors/google_drive



IMPLEMENTED

Private mode

Zero retention: bytes, chunks and vectors live for one request, then are discarded with the answer.

IMPLEMENTED

Import mode

Creates a normal source, so Ask Memora and Graph RAG can use it later.

PARTIAL

Changes in Drive

Changed or vanished files are detected, but a re-import adds a new source beside the old one.

03 · FEATURES

The operator console

The console is the operator's view of everything above: a rail that follows the ownership chain, then processing, retrieval, Graph RAG and connectors.

Overview
Dashboard: live counts and growth

Ownership chain
Tenants · Applications · Actors · Subjects · Sources

Access
API credentials

Processing
Chunks · Embeddings

Retrieval
Ask Memora

Graph RAG
Arena · Benchmarks · Graph explorer · Source inspector

Connectors
Google Drive

Elsewhere
Docs (public site) · API /docs (OpenAPI)

Tenant switcher
The top bar scopes every list and pre-fills every form with one tenant, or shows all of them.

Light and dark
One token file reskins the console; status colours stay muted so badges stand out.

Ownership on every page
A source's page shows its full chain, because it can never be reached by ID alone.

03 · FEATURES

Evaluation harness

A self-contained package in backend/evals measures retrieval and answers on real questions. It only reads the database.

01 **Generate**
Sample chunks; a model drafts questions. Dry-run shows the cost first.

02 **Review**
Approve, reject or edit each question interactively.

03 **Run**
Approved questions go through the real pipeline stages.

04 **Compare**
Diff a new report against a baseline.

Ranking	Context	Honesty	Cost
Recall@5/10	**Precision**	**Refusals**	**p50 / p95**
MRR · nDCG@10 · hit rate	and recall	false answers on unanswerables	latency, spend per run

Lost at stage

Shows where the right chunk dropped out: top-50 search, MMR or the reranker.

Judge scores

Optional, and always labelled as LLM-judged. A separate script checks the judge against your own grades.

PRIVACY

Eval data and reports are git-ignored: they come from real documents.

04 · USER WORKFLOWS

Getting started as an integrating application

04

From an empty console to an application asking questions. An operator does the first three steps once; the application does the rest over the API.

01 Sign in
Google sign-in to the console.

02 Tenant & application
Create the organisation and the consuming system.

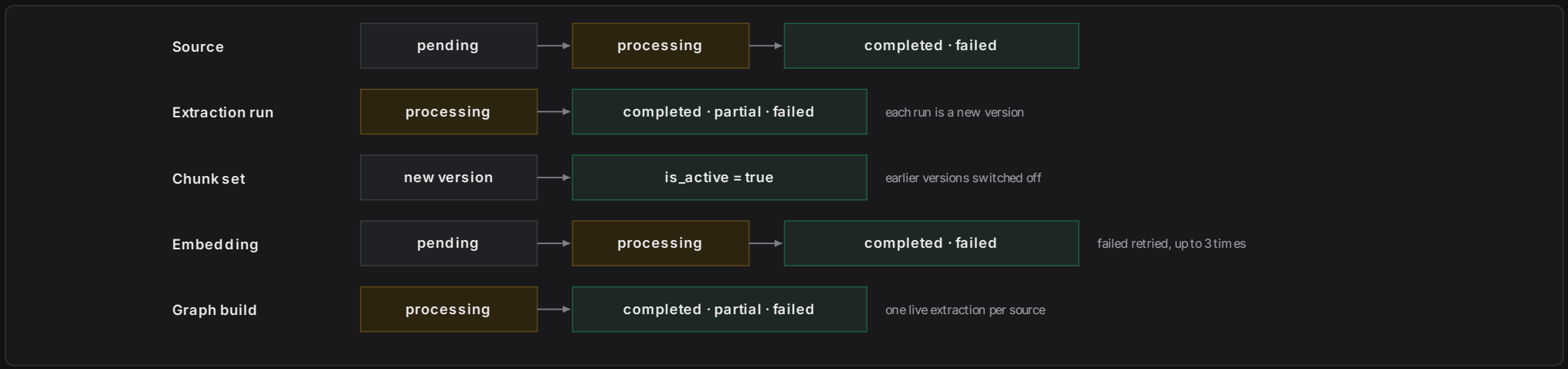
03 API credential
Copy the `memora_…` token; it is shown once.

04 Subject
Create a workspace with your own `external_id`.

05 Upload
`extract=true` reads, chunks and embeds it.

06 Ask
`POST /query` or `/query/stream`.

FIG 10 **Statuses through the stages** schemas/enums.py



05 · ARCHITECTURE

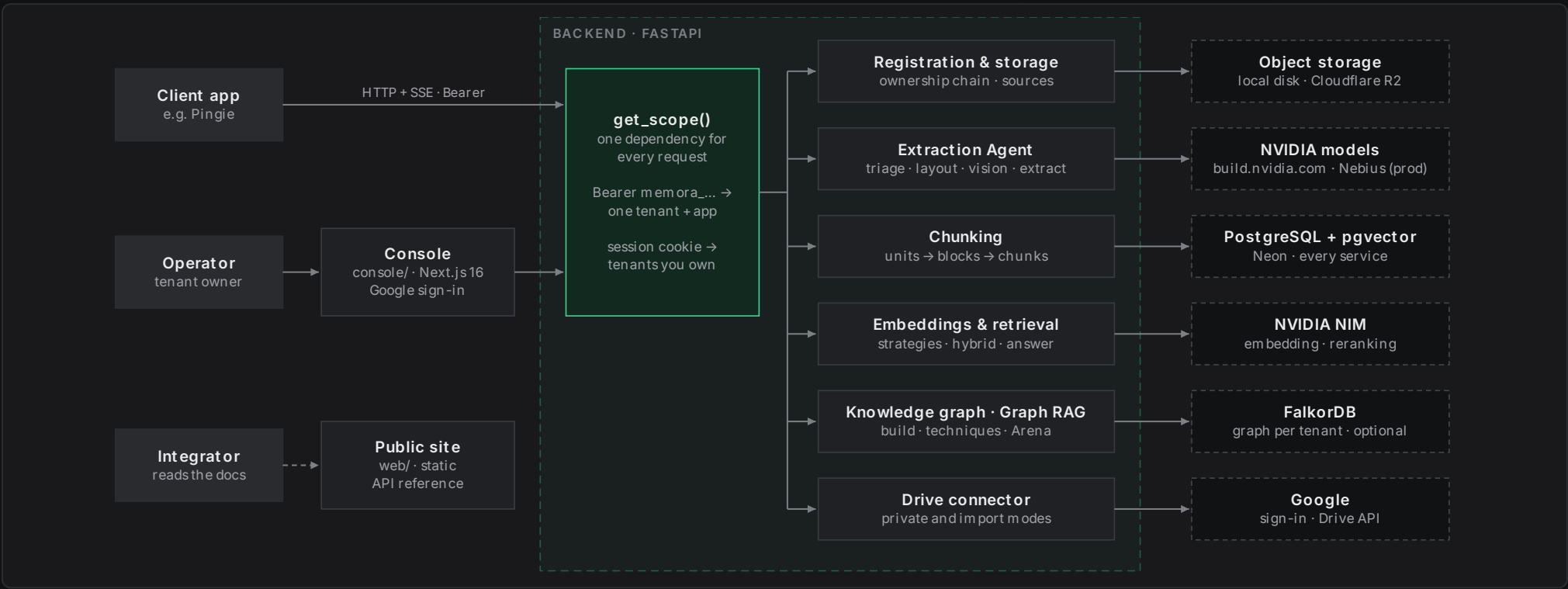
System map

05

Three deployables (backend, console, public site) and the services the backend depends on. Every request is resolved to a scope by one dependency before any service runs.

FIG 11 **Memora system map**

left: callers · centre: backend · right, dashed: outside the repository



KEY TAKEAWAY

Routes stay thin; all business logic and ownership checks live in services, so a new endpoint cannot skip them.

OPERATIONS NOTE

Slow work runs as background tasks in the API process by default; a durable job table with separate workers is a setting away.

05 · ARCHITECTURE

Component responsibilities

What each part owns. Code in the repository first, then the services Memora depends on.

backend
API · PYTHON

Ownership chain, storage, extraction, chunking, embeddings, retrieval, graph, Graph RAG, Drive and jobs.

FastAPI · SQLAlchemy 2 · Alembic

console
OPERATORS

A plain API client: every page fetches through one typed client, with shared loading and error states.

Next.js 16 · React 19 · Tailwind 4

web
PUBLIC

Landing page and static API reference, readable without an account; makes no API calls.

Next.js 16 · static

evals
BACKEND/EVALS

Question sets, metrics and reports over the real pipeline; read-only against the database.

Python CLI

PostgreSQL
EXTERNAL · TRUTH

Every table, vectors via pgvector with HNSW, lexical search via tsvector, the job queue.

Neon in practice

FalkorDB
EXTERNAL · OPTIONAL

One graph per tenant, derived from PostgreSQL and rebuilt from it.

Cypher

NVIDIA models
EXTERNAL · MODELS

Layout, vision, extraction, graph, answers, embeddings and reranking; Nebius serves text models in production.

OpenAI-compatible API

Object storage
EXTERNAL · BYTES

Original files: local disk in development, Cloudflare R2 in production.

StorageBackend protocol

get_scope()
BACKEND/APP/API/DEPS.PY

The single place a token or session becomes a scope. No endpoint validates a token itself.

Tests
BACKEND/TESTS

pytest against a real PostgreSQL, never SQLite; providers are faked so tests spend no credits.

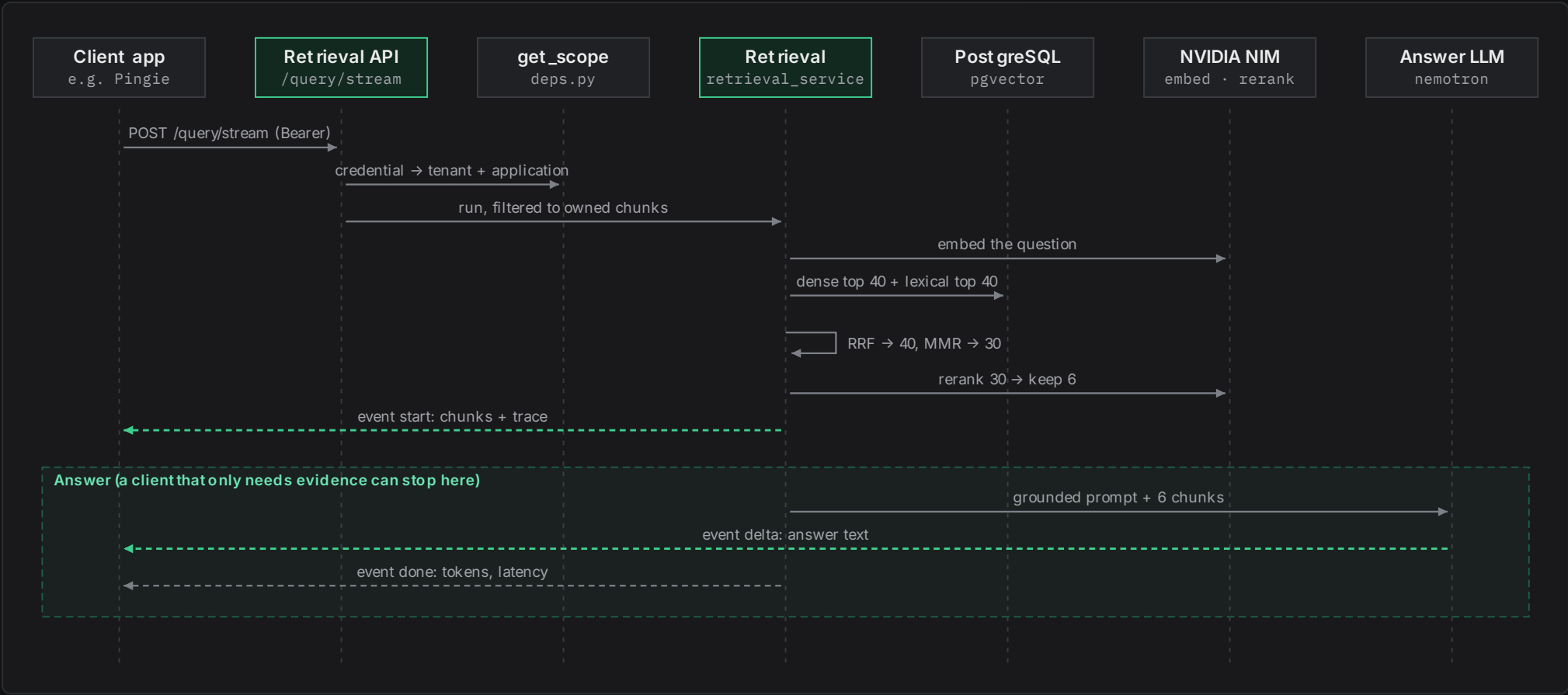
05 · ARCHITECTURE

Sequence: an application asks a question

The streaming endpoint sends evidence before the answer. A client that writes its own answer, such as Pingie, can stop after the first event.

FIG 12 **An application asks a question**

routes/retrieval.py → retrieval_service



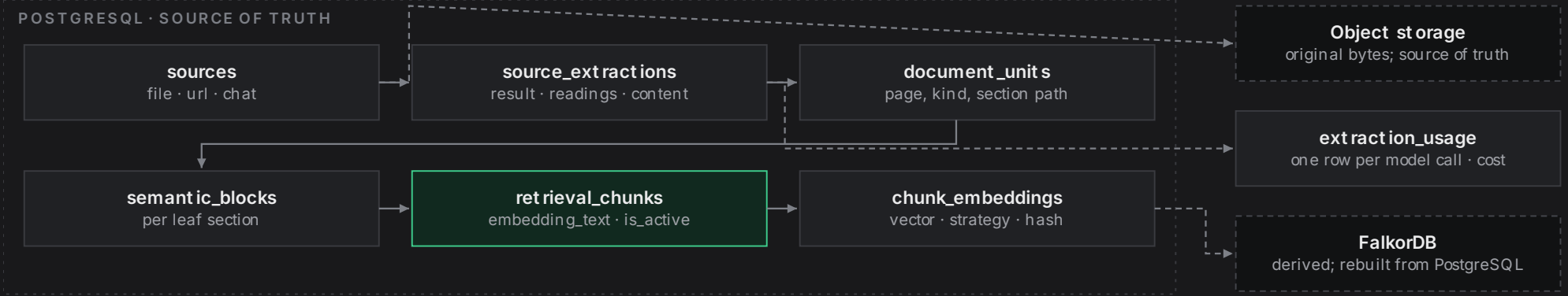
06 · DATA FLOWS & INTEGRATIONS

How information is stored

06

PostgreSQL is the source of truth. Each stage writes its own table, so any later stage can be rebuilt without repeating the earlier ones.

FIG 13 **Data in creation order** docs/data-flow.md · db/models



Three kinds of extraction output

`result` for API consumers, `readings` for chunking, `content` for the graph.

One cost ledger

`extraction_usage` records every model call, failed ones too, for extraction, graph and judge.

Derived stores repair themselves

Graph writes follow the database commit; startup sweeps clear interrupted runs and orphans.

06 · DATA FLOWS & INTEGRATIONS

Integrations and environments

Memora uses NVIDIA open models only: every configured model id must start with nvidia/. The environment setting decides who serves them and where files live.

	LOCAL (DEFAULT)	PRODUCTION
Text models: extract, graph, answer	build.nvidia.com	Nebius Token Factory
Layout, vision, embeddings, reranker	build.nvidia.com	build.nvidia.com (Nebius has none)
File storage	Local disk	Cloudflare R2

PostgreSQL + pgvector

Required. All state, vectors, lexical index and jobs.

FalkorDB

Optional. Without it, graph endpoints answer 503.

Google

Console sign-in, and Drive with a read-only scope.

Pricing file

Model prices in `pricing.json` turn token counts into cost.

KEY TAKEAWAY

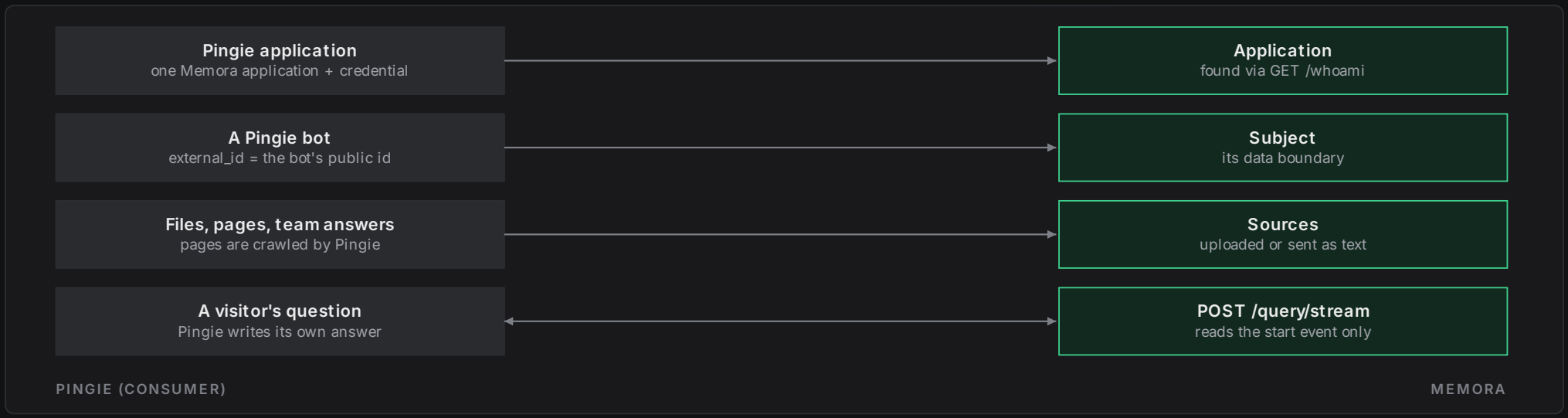
Both model providers speak the OpenAI API, so switching environments changes only configuration. Each call's cost is recorded under the provider that served it.

06 · DATA FLOWS & INTEGRATIONS

Integration example: Pingie on Memora

Pingie, an AI support chatbot, uses Memora purely for retrieval. Its own reasoning layer decides what to say.

FIG 14 **Pingie on Memora** ping-backend/app/clients/memora.py



Bot = subject

Each bot is a subject, so one bot's sources never answer another's visitors.

Evidence, not answers

Pingie keeps the start event's ranked chunks and writes the reply with its own model.

Chat memory

Finished chats a teammate answered are pushed back as sources for later retrieval.

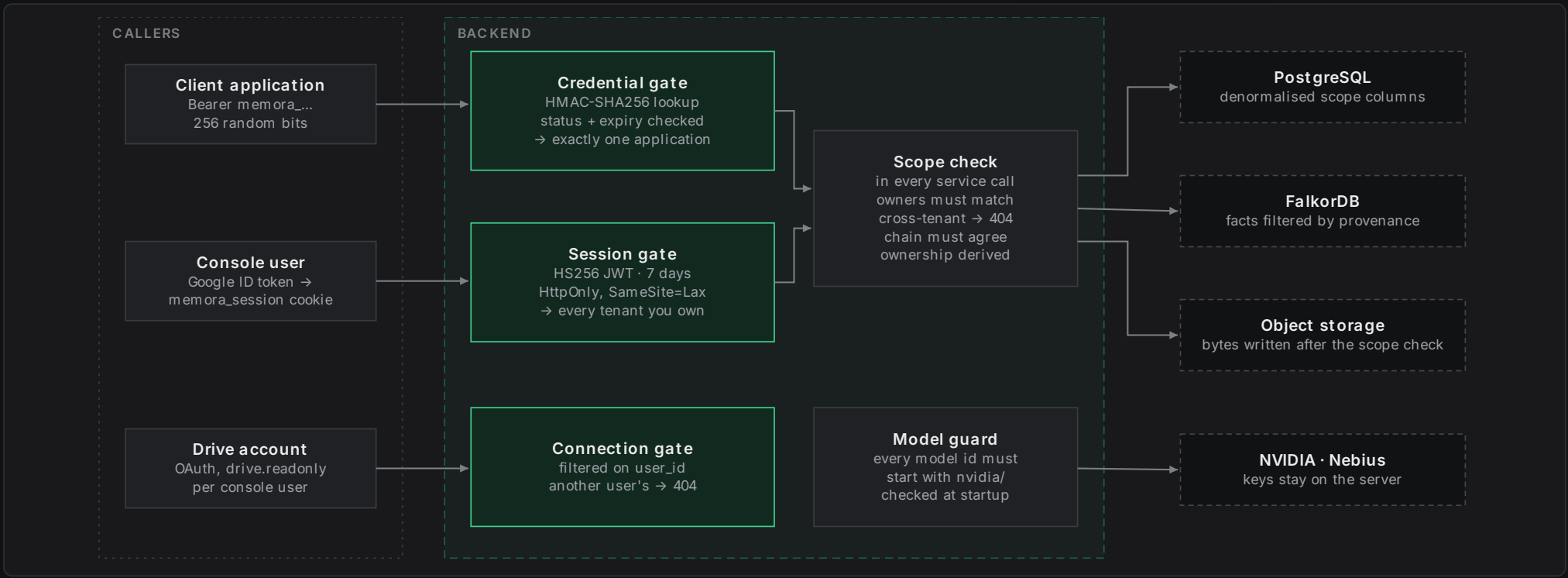
07 · SECURITY & PERMISSIONS

Trust boundaries

07

Three kinds of caller pass three gates into the same scope check. Behind it, every service compares a row's owners with the caller's scope.

FIG 15 **Trust boundaries**



KEY TAKEAWAY

Cross-tenant reads return 404, not 403: a 403 would confirm the ID exists in someone else's tenant.

KNOWN LIMIT

A console user's scope is every tenant they own. There are no per-actor permissions inside a subject yet.

07 · SECURITY & PERMISSIONS

Security controls

Controls verified in the backend, and the ones still missing.

Hashed API tokens

IMPLEMENTED

Only an HMAC-SHA256 keyed by a server secret is stored; the raw token is shown once.

Google-only console sign-in

IMPLEMENTED

ID token verified with Google; 7-day HttpOnly JWT cookie.

Scope on every call

IMPLEMENTED

Ownership derived, chain checked, 404 for anything outside scope.

Per-user Drive access

IMPLEMENTED

Connections are filtered by user; private mode stores nothing.

Why a keyed HMAC, not bcrypt

Authentication must find the credential from the token alone, which needs an indexed lookup; a per-row salt would force a scan. Tokens are 256 random bits, so slow hashing adds nothing, and a database dump can't forge tokens because the key never touches the database. Rotating it invalidates every credential.

Gaps found in the repository

- Per-actor permissions inside a subject (`subject_actors`). — NOT BUILT
- Upload file-type allow-list. — NOT BUILT
- The session secret falls back to a placeholder if unset; set it in production. — PARTIAL
- The README still describes a removed admin-key login. — PARTIAL

08 · LIMITATIONS & SCOPE

08 What is built, partial, or not built

Every difference found between the code and the repository's own descriptions, in one place.

IMPLEMENTED — Built and reachable

- Ownership chain, folders, credentials
- Upload, extraction, chunking, embeddings
- Hybrid retrieval and streamed answers
  the README's 50 → 10 → 5 sizes are out of date
- Knowledge graph, Arena, benchmarks, Advisor
- Drive private and import modes
- Evaluation harness, durable job mode

PARTIAL — Narrower than described

- Unified Graph RAG
  API only; not in the console Arena
- Drive resync
  detects changes; re-import duplicates the source
- Cost cap
  meters graph extraction only
- Graph paths
  an out-of-scope edge drops a path instead of finding another

NOT BUILT — Not built

- Per-actor permissions in a subject
- Memory Agent and multi-turn conversation memory
- LLM-generated Cypher
- Drive formats other than PDF
- Deleting a tenant
  left out on purpose: cascades are too destructive

EVIDENCE NOTES

- Results from the unified pipeline come from one corpus of 15 sources; treat them as indications.
- The design records in docs/ are deleted in the working tree but present at f4c6b8c; this handbook read them from git.

09 · QUICK REFERENCE

09 Quick reference

The numbers people ask about most, with their defaults from the backend configuration.

Upload size **50** MiB any file type	Pages per extraction **30** 4 model calls in parallel	Chunk target **300–500** tokens · 1,200 hard max	Embedding **2,048** dims nemotron-3-embed-1b
Candidates **40 + 40** dense + lexical, RRF k 60	After MMR **30** λ 0.7, then reranker	Final context **6** chunks to the answer model	Weak evidence **−5.0** best rerank score (Graph RAG)
Graph units **3,000** chars 300 overlap	Session **7** days memora_session cookie	Drive private **10 / 100** files / pages per question	Job attempts **3** 30-minute lease

Main endpoints

`POST /subjects/{id}/sources/upload` · `POST /sources/{id}/extractions` · `POST /query` · `POST /query/stream` · `POST /compare/query` · `GET /whoami`

Local ports

backend `:8000` · console `:3000` · public site `:3001` · API docs `/docs`

09 · QUICK REFERENCE

Flashcards

Check your understanding. Each answer fits on one line.

Q01

What is the difference between an actor and a subject?

A The actor caused it; the subject is the workspace.

Q02

Why does a cross-tenant read return 404?

A A 403 would confirm that the ID exists.

Q03

Which pages go to the layout model?

A Hard ones: scans, tables, or many images.

Q04

Why is chunking free?

A It runs locally on stored readings; no model call.

Q05

What makes an embedding strategy safe to change?

A It can't change: a new setup is a new strategy.

Q06

Which two searches does a question run?

A Dense HNSW and lexical tsvector, fused by RRF.

Q07

How many chunks reach the answer model?

A Six, after MMR and the reranker.

Q08

How does Graph RAG know nothing relevant was found?

A The best rerank score is below −5.0.

Q09

What does Drive private mode keep?

A Nothing: it is discarded with the answer.

Q10

Why are API tokens hashed with HMAC?

A Indexed lookup, and a dump can't forge tokens.

Q11

What happens without FalkorDB?

A Graph endpoints answer 503; the rest works.

Q12

How does Pingie use Memora?

A Retrieval only: it reads the stream's start event.

09 · QUICK REFERENCE

Glossary

Terms used in the console, the API and this handbook.

Tenant
The organisation that owns the data; top of the chain.

Application
A consuming system inside a tenant; holds credentials.

API credential `memora_…`
A token that resolves to one application.

Actor
Who or what caused something; audit only.

Subject
A workspace and data boundary; nests as folders.

Source
A registered file, URL or chat.

Extraction
A versioned reading of a source, with its cost.

Triage
Labelling each page easy, medium or hard.

Reading
One page's Markdown transcription.

Document unit
A heading, paragraph, list, table or figure.

Semantic block
Units grouped under one leaf section.

Retrieval chunk
The final piece that is embedded and searched.

Strategy
An immutable way of embedding chunks with a model.

HNSW
The approximate nearest-neighbour index in pgvector.

RRF
Reciprocal rank fusion of several ranked lists.

MMR
Maximal marginal relevance: diverse, relevant picks.

Reranker
A cross-encoder that scores question–chunk pairs.

Ontology
The allowed entity and relation types for the graph.

Graph RAG
Retrieval that uses the knowledge graph for evidence.

Arena
Console page comparing pipelines on one question.

Atomic fact
One self-contained sentence with its quote (unified).

Community
A cluster of related entities, with a summary.

Scope
What a request may see, resolved by get_scope().

Private mode
Drive Q&A with nothing stored.